Lab 3 Report: Proportional-Derivative Wall Follower

Team 9

Allen Chen
Simdi Okoroafor
Soe Wai Yan

Robotics, Science Systems

March 13, 2026

1 Introduction

Authored by Simdi Okoroafor

Developing an autonomous vehicle requires combining perception and control so that the system can interpret its environment and respond with stable driving behavior. In this lab, we designed and implemented a wall-following controller that allows the vehicle to maintain a desired distance from a wall using LiDAR data. This lab builds on previous RSS labs that focus on working with ROS2, sensor data, and vehicle actuation, and it represents an important step toward building a fully autonomous driving system.

The purpose of this lab is to design an algorithm that uses LiDAR measurements to estimate the position of a nearby wall and generate steering commands that keep the vehicle at a specified distance from it. The system must operate in real time, continuously processing incoming LiDAR scans and updating the steering commands as the vehicle moves through the environment.

The core technical challenge is converting raw LiDAR range measurements into a reliable estimate of the vehicle's position relative to the wall. Our solution uses geometric relationships between selected LiDAR rays to estimate the wall's orientation and predict the vehicle's distance from the wall at a lookahead point.

This predicted distance is then used in a Proportional–Derivative (PD) controller that computes steering commands to minimize the distance error and maintain stable wall-following behavior.

2 Technical Approach

2.1 Wall Following

Authored by Simdi Okoroafor

The wall follower node was implemented as a reactive control algorithm that processes LiDAR scan data and generates steering commands to maintain a constant distance from a wall. The approach converts raw LiDAR measurements into a geometric estimate of the vehicle's position relative to the wall, and then uses this estimate in a feedback controller to compute steering commands in real time.

1. **Ray Selection and Filtering**

   To estimate the wall geometry, the algorithm uses two LiDAR measurements taken at different angles relative to the vehicle. One ray is chosen approximately perpendicular to the wall (around 90° relative to the vehicle), and the second ray is selected at a forward-facing angle (around 55°). These two measurements allow the algorithm to estimate the orientation of the wall relative to the vehicle.

   Rather than using a single LiDAR reading, which can be noisy, a small window of neighboring rays around each selected angle is taken and the median value is computed. This median filtering helps reduce the influence of noise, outliers, or spurious reflections in the LiDAR data, resulting in a more stable distance estimate.

2. **Wall Geometry Estimation**

   Using these two filtered distance measurements, the algorithm computes the wall angle and estimates the current perpendicular distance from the wall.

3. **Lookahead Prediction**

   A lookahead distance is then calculated using the wall angle and a fixed lookahead parameter. This predicts the vehicle's distance from the wall slightly ahead along its trajectory, allowing the controller to anticipate future deviations instead of reacting only to the current position.

4. **Control Computation**

   The predicted distance is compared with the desired wall distance to compute the control error. A Proportional–Derivative (PD) controller converts

this error into a steering command, where the proportional term corrects present error and the derivative term dampens oscillations.

5. **Wall Side Handling and Safety Limits**

   The node supports following either the left or right wall, which is handled by adjusting the sign of the distance error depending on the selected side. Finally, steering commands are clamped within a safe range, and vehicle speed is fixed at a stable value to prevent overly aggressive motion. These design choices allow the system to follow the wall smoothly and robustly using real-time LiDAR data.

2.2 Safety Node

Authored by Allen Chen

The main goal for safety control is to ensure the car does not cause collateral damage while in operation, especially at high speeds. At the same time, we don't want the safety to be so obstructive that it stops the autonomous vehicle from accomplishing its driving tasks.

We decided to implement a simple distance threshold trigger in which the car would brake if it detected that it was within the specified threshold with respect to the closest object. This ensures that we would be able to specify the distance at which we want to stop and tune the stopping distance so that it stops before crashing into the object, while also making sure it does not pick up farther objects as obstacles. The algorithmic overview below shows how the safety algorithm is triggered and exited.

1. Gets Lidar scan ranges from /scan topic from angles -30 to +30 degrees.
2. Finds the smallest range, minrange.
3. If minrange is less than the distance threshold, dist_thresh, publish the stop command to /safety, which overrides /ackermann_cmd, where the wall follower drive commands are published.
4. If /scan keeps on detecting obstacles within dist_thresh, keep on sending stop command to /safety.
5. If no /scan ranges are within dist_thresh, stop sending stop command.

3 Experimental Evaluation

3.1 Wall Follower Testing

Authored by Soe Wai Yan

To evaluate the performance of the wall-following controller, we analyze several test cases using an error metric. The error represents the difference between the desired distance and the actual distance from the wall. In our experiments, the desired distance is set to 1.5 m to maintain a safe separation from the wall.
The error is defined as

$$e = d_{\text{desired}} - d_{\text{actual}}$$

where d_{desired} is the target distance from the wall and d_{actual} is the measured distance from the LiDAR sensor.
A positive error indicates that the vehicle is too close to the wall, while a negative error indicates that the vehicle is too far from the wall.

3.1.1 Test Case 1: Straight Wall Following at 1 m/s

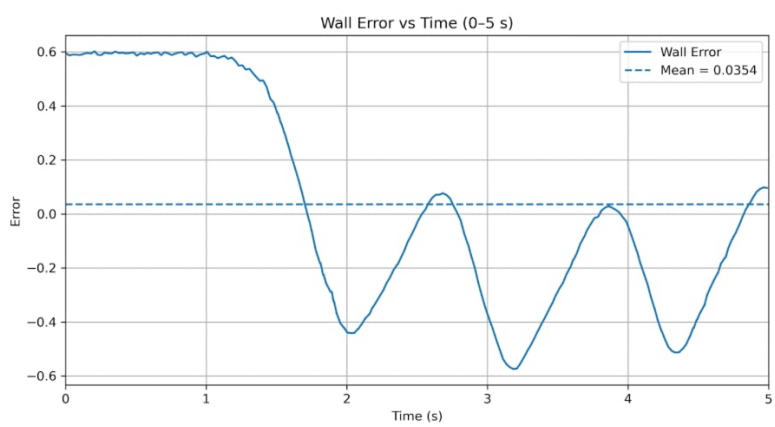

Figure 1: Wall-following error over time for straight wall tracking at 1 m/s.

In this test case, the vehicle followed a straight wall at a speed of 1 m/s while starting from the right side. At the beginning, the error was relatively large because the vehicle started farther from the desired distance. As the controller responded to the LiDAR measurements, the error quickly decreased and stabilized near the desired value. The mean error was approximately 0.035 m, which indicates that the controller maintained a fairly accurate distance from the wall. Small oscillations were still present, suggesting that further tuning of the P and D gains could improve smoothness.

3.1.2 Test Case 2: Straight Wall Following at 3 m/s

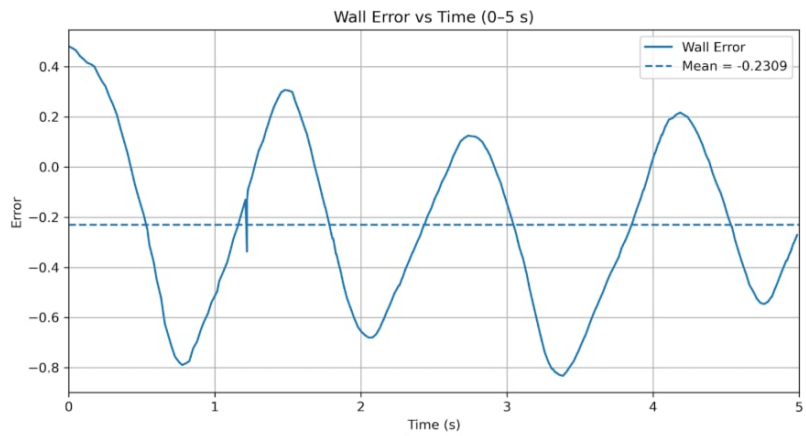

Figure 2: Wall-following error over time for straight wall tracking at 3 m/s.

In this test case, the vehicle followed the same straight wall but at a higher speed of 3 m/s. Compared to the previous test, the error showed larger oscillations. This occurred because the higher speed reduced the time available for the controller to react to changes in the wall distance. The mean error was approximately 0.23 m which means the vehicle tended to stay slightly farther from the wall than the desired distance. This result suggests that the controller gains work better at lower speeds and may require additional tuning for higher-speed operation.

3.1.3 Test Case 3: Starting from the Middle of the Corridor

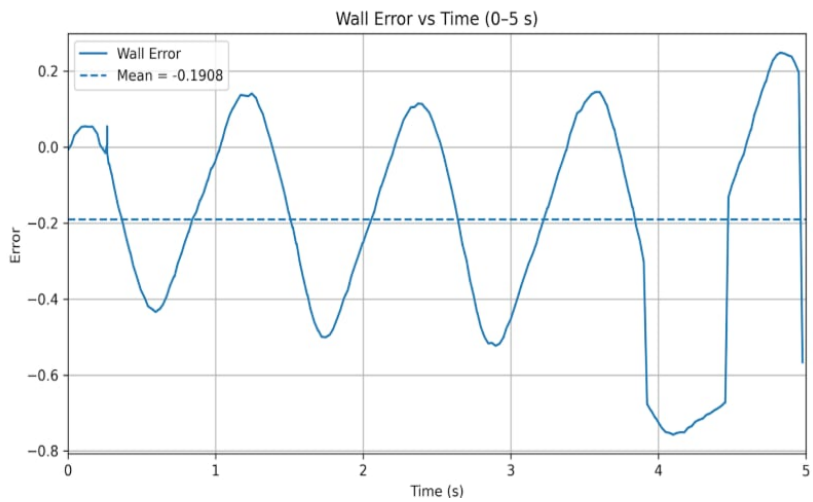

Figure 3: Starting from the Middle of the Corridor at 1 m/s.

In this test case, the vehicle started in the middle of the corridor at a speed of 1 m/s. Since the vehicle began far from the wall, the initial error was large. The controller then steered the vehicle toward the wall and gradually reduced the error. Over time, the vehicle stabilized near the desired distance, and later the controller was able to recover from an initially large deviation.

3.1.4 Test Case 4: Outer Corner Turn

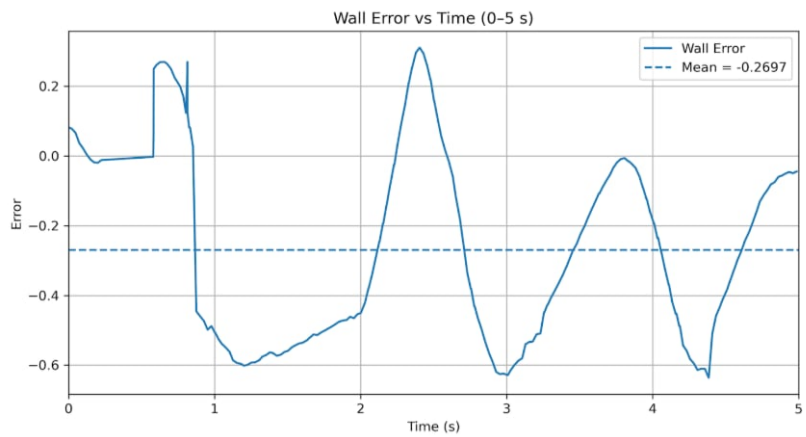

Figure 4: Outer Corner Turn at 1 m/s.

In this test case, the vehicle followed the wall while turning around an outer corner at a speed of 1 m/s. When the vehicle reached the corner, the wall distance changed suddenly, which caused a spike in the error in the graph. The controller then adjusted the steering to recover and continue following the wall. After completing the turn, the error oscillated for a short time as the vehicle stabilized again near the desired distance.

3.1.5 Test Case 5: Inner Corner Turn

In this test case, the vehicle attempted to follow the wall through an inner corner at a speed of 1 m/s. Unlike the outer corner scenario, the controller struggled because the wall temporarily disappeared from the LiDAR view when the vehicle reached the corner. This caused a large spike in the error and the controller lost the correct wall reference. As a result, the vehicle was unable to maintain stable wall-following behavior through the inner corner. To improve this performance, a wall-loss recovery strategy could be implemented such as increasing the look-ahead distance or expanding the sensing angle range.

3.1.6 Test Case 6: Starting Facing Away from the Wall

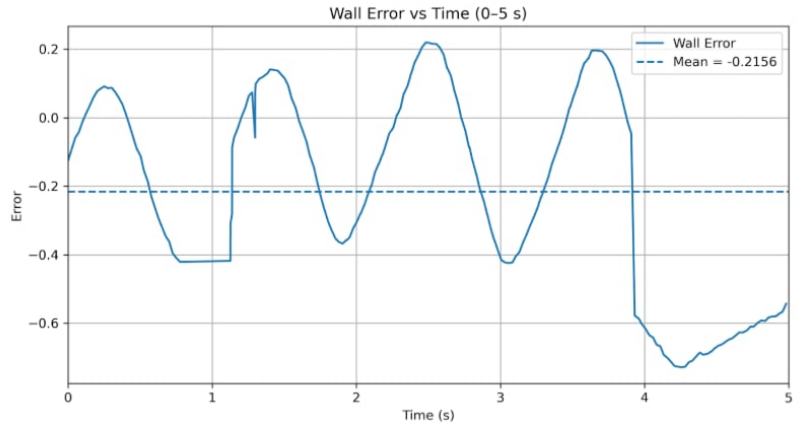

Figure 5: Starting Facing Away from the Wall at 1 m/s.

In this test case, the vehicle started facing away from the wall at a speed of 1 m/s. Since the wall was not initially within the main sensing region of the LiDAR, the controller first needed to detect and reacquire the wall. As the vehicle moved forward, the LiDAR began to detect the wall again and the controller steered the vehicle toward the desired distance. After this recovery, the vehicle resumed normal wall-following behavior.

3.2 Safety Controller Tests

Authored by Allen Chen

For our experiment, we needed to determine if the use of a braking distance would satisfy the safety requirement at different speeds. As such, we had the car run at a wall in front of it at speeds of 1, 1.5, 2, 2.5 m/s and we kept on increasing the distance threshold at 0.1 m/s until the car was able to stop successfully. We then recorded the Stopping Distance Error, SDE, for successful stopping cases to see how much rolling there is after braking at different speeds. A successful stopping case is one where the car stops before touching the wall.

$$SDE = dist_thresh - minrange$$

where dist_thresh is the specified distance from the closest detected obstacle the car starts braking and minrange is the assumed distance from wall to Lidar sensor, which is also the smallest range the Lidar detects. This allows us to see how much the car rolls after braking.

We didn't test higher speeds due to the max limit of the car speed being 2.5 m/s.

3.2.1 Braking at 1 m/s

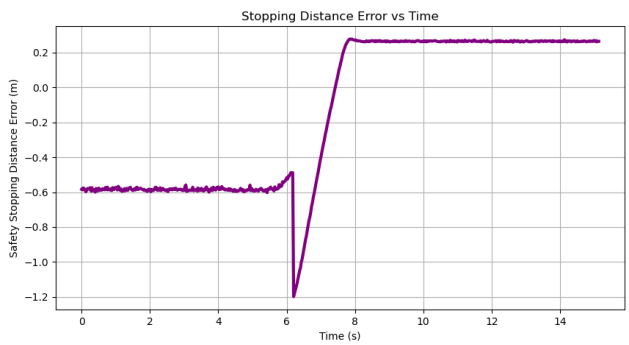

Figure 6: Graph of the SDE vs. time stopping from car speed at 1 m/s. The graph shows that braking with set distance is very risky as the car stops 60% into the buffer between the threshold distance and the wall. The threshold on the graph is crossed at SDE = 0 m at 7.5 s, meaning the car rolled 0.30 m.

For a car traveling at constant speed of 1 m/s, the minimum *dist_thresh* needed is 0.5 m. This caused the car to roll about 60% into the buffer between the threshold and wall, leaving about 0.2 m between sensor and wall.

3.2.2 Braking at 1.5 m/s

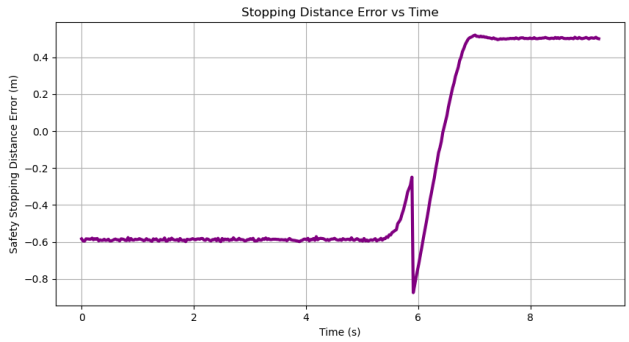

Figure 7: Graph of the SDE vs. time stopping from car speed at 1.5 m/s. The graph shows that braking with set distance is very risky as the car stops 71.4% into the buffer between the threshold distance and the wall. The threshold on the graph is crossed at SDE = 0 m at 6.5 s, meaning the car rolled 0.5 m.

For a car traveling at constant speed of 1.5 m/s, the minimum *dist_thresh* needed is 0.7 m. This caused the car to roll about 71.4% into the buffer between the threshold and wall, leaving about 0.20 m between sensor and wall.

3.2.3 Braking at 2 m/s

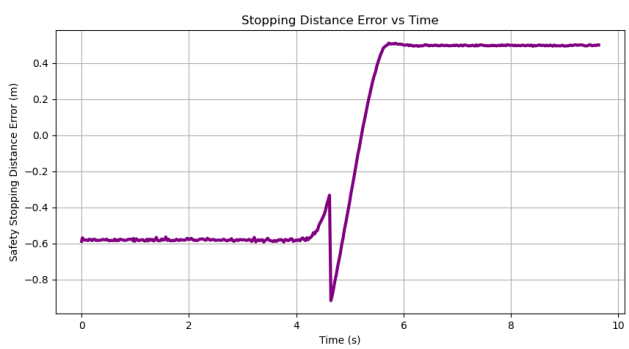

Figure 8: Graph of the SDE vs. time stopping from car speed at 2 m/s. The graph shows that braking with set distance is more acceptable as the car stops 45.5% into the buffer between the threshold distance and the wall. The threshold on the graph is crossed at SDE = 0 m at 5.2 s, meaning the car rolled 0.5 m.

For a car traveling at constant speed of 2 m/s, the minimum *dist_thresh* needed is 1.1 m. This caused the car to roll about 45.5% into the buffer between the threshold and wall, leaving about 0.6 m between sensor and wall.

3.2.4 Braking at 2.5 m/s

Figure 9: Graph of the SDE vs. time stopping from car speed at 2.5 m/s. The graph shows that braking with set distance is very risky as the car stops 76.9% into the buffer between the threshold distance and the wall. The threshold on the graph is crossed at SDE = 0 m at 3.0 s, meaning the car rolled 1.0 m.

For a car traveling at constant speed of 2.5 m/s, the minimum *dist_thresh* needed is 1.3 m. This caused the car to roll about 76.9% into the buffer between the threshold and wall, leaving about 0.3 m between sensor and wall.

4 Conclusion

Authored by Soe Wai Yan and Allen Chen

To conclude, we successfully implemented a LiDAR-based wall-following controller using PD control. The vehicle was able to track the wall relatively well, especially at lower speeds, maintaining a small average error. Through our experiments, we also learned that controller tuning plays a major role in stability. When the vehicle speed increased, we observed larger oscillations, showing that the current gains are better suited for lower speeds. For future work, we plan to implement a wall-loss recovery strategy so the vehicle can handle situations like inner corner turns where the wall temporarily disappears.

In addition, the graphs from safety controller tests showed that braking at a set distance threshold is sufficient if the car was always traveling at a constant speed, but for more varied speeds, it is difficult to set a distance threshold as farther thresholds at low speeds invite detection of non-target objects, while

shorter thresholds at high speeds lead to horrible collisions. This strongly suggests the use of using speed as a variable in setting braking distances as there is a general trend in speed vs. successful braking distances.

5 Lessons Learned

One of the main lessons from this lab was how important tuning is, especially when transitioning from simulation to real hardware. In simulation, the LiDAR data is very clean and predictable, which makes it easier for the controller to behave well with a wide range of parameter values. However, when running the algorithm on the real robot, the sensor data was noticeably noisier and sometimes contained inconsistent readings. Because of this, parameters that worked well in simulation did not always perform well on the physical system. This made it clear that real-world systems require more careful tuning and additional considerations for sensor noise. Techniques such as taking the median of a small window of LiDAR rays helped make the distance estimates more stable and improved the overall performance of the controller.
Another lesson from this lab was the importance of communication when working on code that needs to integrate with other components. In this lab, different pieces of software often depend on each other. So, clearly communicating changes to code, assumptions about inputs and outputs, and testing results helps ensure that all parts of the system work together correctly. Good communication within the team therefore plays a key role in making sure the final system behaves as expected.

6 Algorithms Used

6.1 Wall Follower

Authored by Simdi Okoroafor

Input: LiDAR scan ranges r, desired wall distance d_{des}, lookahead distance L, wall side $s \in \{\text{left}, \text{right}\}$

Output: Steering command δ and speed command v

if $s = \textit{left}$ **then**

  set $\theta_a \leftarrow 90^\circ$, $\theta_b \leftarrow 55^\circ$;

else

  set $\theta_a \leftarrow -90^\circ$, $\theta_b \leftarrow -55^\circ$;

end

select a small window of LiDAR rays around θ_a and θ_b;

compute median distances: $a \leftarrow \text{median}(r \text{ near } \theta_a)$, $b \leftarrow \text{median}(r \text{ near } \theta_b)$;

compute angular separation: $\theta \leftarrow |\theta_a - \theta_b|$;

compute wall angle: $\alpha \leftarrow \tan^{-1}\left(\dfrac{a\cos\theta - b}{a\sin\theta}\right)$;

compute current perpendicular distance: $D_t \leftarrow b\cos\alpha$;

compute lookahead distance: $D_{t+1} \leftarrow D_t + L\sin\alpha$;

if $s = \textit{left}$ **then**

  $e \leftarrow d_{\text{des}} - D_{t+1}$;

else

  $e \leftarrow D_{t+1} - d_{\text{des}}$;

end

compute derivative term: $\dot{e} \leftarrow \dfrac{e - e_{\text{prev}}}{\Delta t}$;

compute PD steering command: $\delta \leftarrow K_p e + K_d \dot{e}$;

clamp steering: $\delta \leftarrow \text{clamp}(\delta, -\delta_{\max}, \delta_{\max})$;

set speed: $v \leftarrow v_{\text{fixed}}$;

publish Ackermann drive command (v, δ);

store current error: $e_{\text{prev}} \leftarrow e$;

Algorithm 1: Wall Follower Algorithm

Input: LiDAR scan r, scan angles ϕ, latest drive command (v, δ),
range limits $r_{\min}, r_{\max}$, stop threshold d_{stop}, hold time T_{hold}
Output: Safe drive command $(v_{\text{safe}}, \delta_{\text{safe}})$
select LiDAR rays in the forward cone: $\phi \in [-30°, 30°]$;
discard invalid ranges outside $[r_{\min}, r_{\max}]$;
if *no valid rays remain* **then**
  return;
else
  $d_{\min} \leftarrow \min(r)$;
end
compute safety error: $e_{\text{safety}} \leftarrow d_{\text{stop}} - d_{\min}$;
publish e_{safety};
if $d_{\min} < d_{stop}$ **then**
  braking $\leftarrow$ true;
  $t_{\text{release}} \leftarrow \max(t_{\text{release}},\ t_{\text{now}} + T_{\text{hold}})$;
end
if *braking* $=$ *true and* $t_{now} \geq t_{release}$ *and* $d_{\min} \geq d_{stop}$ **then**
  braking $\leftarrow$ false;
end
$v_{\text{safe}} \leftarrow v$, $\delta_{\text{safe}} \leftarrow \delta$;
if *braking* $=$ *true* **then**
  $v_{\text{safe}} \leftarrow 0$;
end
publish $(v_{\text{safe}}, \delta_{\text{safe}})$;

Algorithm 2: Safety Controller Algorithm